Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Βάσεις Δεδομένων

Ονοματεπώνυμα	Θερκιάι Κριστιάν - 03121905 Μονάντερος Φώτιος - 03121912 Μπενάς Γεώργιος - 03122895
Εξάμηνο	6ο
Ομάδα	Ομάδα 29
GitHub Repository:	https://github.com/ntua-el21912/Databases-Project-2025

Περιεχόμενα:

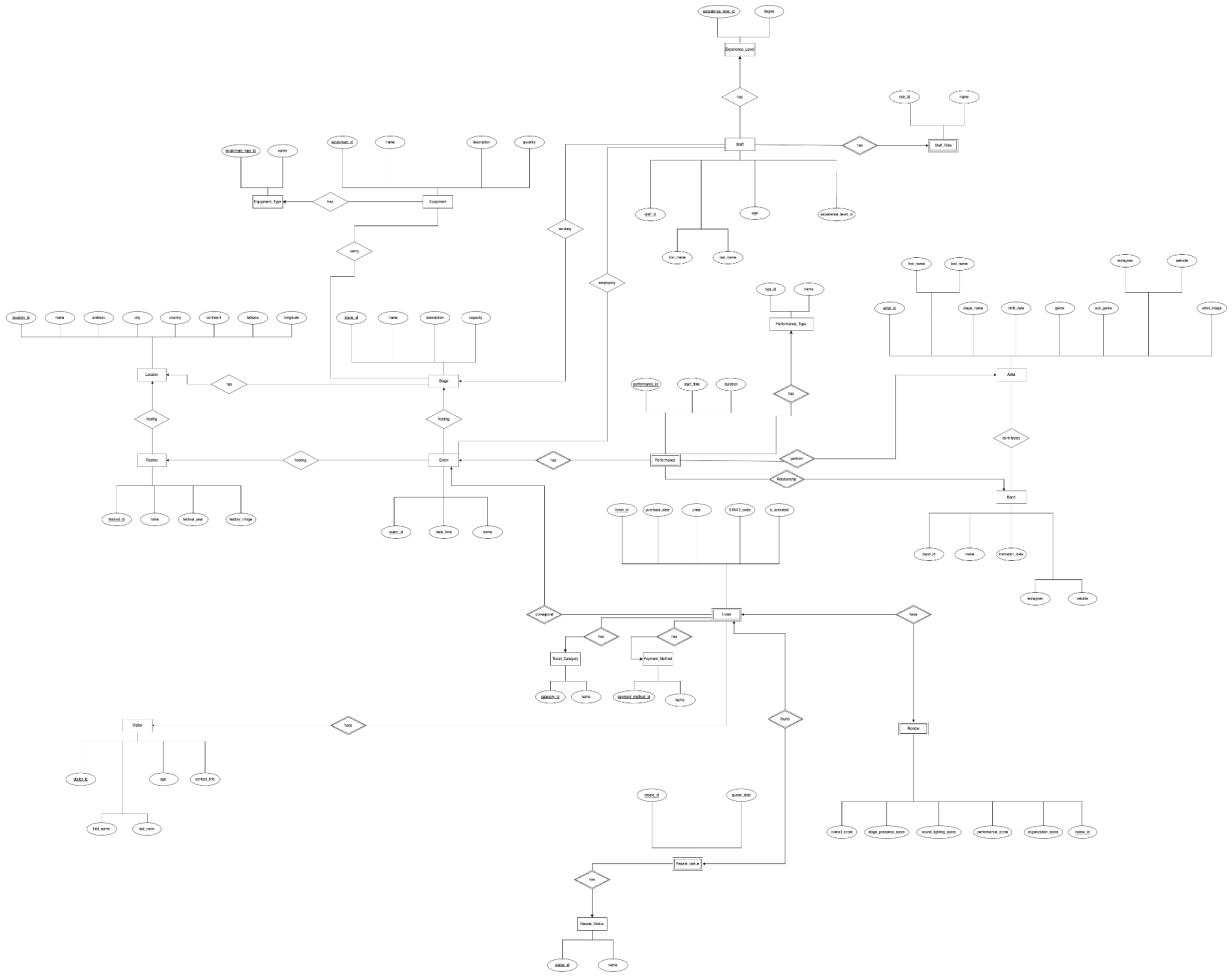
Εισαγωγή **2**
ER Model **2**
Relational Diagram **3**
Πίνακες και Οντότητες 4
Πληθηκότητα και Σχέσεις 5
Indexes 6
DDL **7**
Procedures 24
Triggers 24
Queries 25

Εισαγωγή

Η παρούσα αναφορά παρουσιάζει λεπτομερώς τον σχεδιασμό, την υλοποίηση και την αξιολόγηση του συστήματος βάσης δεδομένων που αναπτύχθηκε στα για την οργάνωση και διαχείριση του φεστιβάλ Pulse University, καλύπτοντας πτυχές όπως χώρους, καλλιτέχνες, παραστάσεις, εισιτήρια και προσωπικό. Παρακάτω αναλύονται οι λογικές και φυσικές δομές της βάσης, οι λειτουργικές απαιτήσεις, τα σύνθετα ερωτήματα καθώς και τεχνικές βελτιστοποίησης αποδοτικότητας.

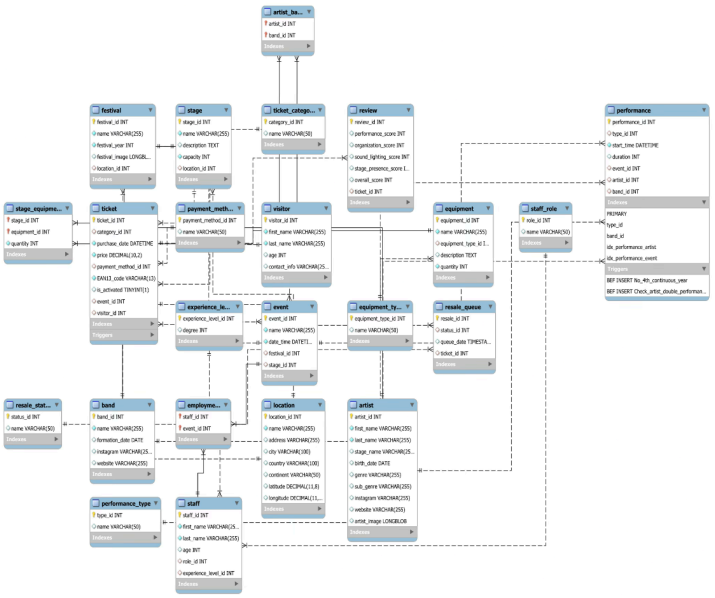
ER Model

Πριν προχωρήσουμε στην υλοποίηση του σχεσιακού διαγράμματος, είναι απαραίτητο να δημιουργήσουμε ένα Διάγραμμα Οντοτήτων-Συσχετίσεων (ER), το οποίο θα μας καθοδηγήσει στη διαδικασία σύνδεσης των οντοτήτων της βάσης δεδομένων μας. Το διάγραμμα αυτό σχεδιάστηκε με το draw.io και παρατίθεται εδώ:



Relational Diagram

Η βάση δεδομένων που αναπτύξαμε αφορά ένα φεστιβάλ, με στόχο να δίνεται στον χρήστη η δυνατότητα να εξάγει στοιχεία σχετικά με την διοργάνωση που τον ενδιαφέρουν. Αυτό περιλαμβάνει τα έσοδα, τις συμμετοχές των καλλιτεχνών, τις εμφανίσεις τους, το προσωπικό, τα μουσικά είδη και την αξιολόγηση που γίνεται από τους επισκέπτες. Στη συνέχεια παρουσιάζονται οι πίνακες που αποτέλεσαν τη βάση για την κατασκευή του σχεσιακού διαγράμματος.



Το Relational Diagram δημιουργήθηκε μέσω του reverse engineer από το MySQL Workbench.

Από ένα σχεσιακό διάγραμμα μιας βάσης δεδομένων αντλούμε σημαντικές πληροφορίες που μας βοηθούν να κατανοήσουμε τη δομή, τη λειτουργία και τις σχέσεις μεταξύ των δεδομένων. Συγκεκριμένα, περιλαμβάνει:

1) <u>Πίνακες</u> (Tables)

Δείχνει όλους τους πίνακες που υπάρχουν στη βάση δεδομένων.

Κάθε πίνακας αντιστοιχεί σε μια οντότητα (entity) του συστήματος.

2) Στήλες (Attributes / Columns)

Παρουσιάζει τα πεδία που περιέχει κάθε πίνακας, δηλαδή τα χαρακτηριστικά της κάθε οντότητας.

Φαίνεται ο τύπος δεδομένων και αν επιτρέπεται ή όχι κενή τιμή (NULL/NOT NULL).

3) Πρωτεύοντα Κλειδιά (Primary Keys)

Υποδεικνύει ποιο πεδίο ή ποιος συνδυασμός πεδίων ταυτοποιεί μοναδικά κάθε εγγραφή σε έναν πίνακα.

4) Ξένα Κλειδιά (Foreign Keys)

Δείχνει τις σχέσεις μεταξύ των πινάκων, μέσω των ξένων κλειδιών.

Δηλώνει ποια πεδία συνδέουν πίνακες μεταξύ τους.

5) Σχέσεις (Relationships)

Καθορίζει τη φύση της σχέσης: ένα-προς-ένα (1:1), ένα-προς-πολλά (1:N) ή πολλά-προς-πολλά (M:N).

Βοηθά στην κατανόηση του τρόπου που αλληλεπιδρούν τα δεδομένα.

6) Περιορισμοί

Παρουσιάζει κανόνες όπως UNIQUE, CHECK, DEFAULT που περιορίζουν τις τιμές των πεδίων.

Συνοψίζοντας:

Το σχεσιακό διάγραμμα παρέχει μια σαφή και οπτική απεικόνιση της δομής της βάσης δεδομένων, βοηθώντας στον σχεδιασμό, τη συντήρηση, αλλά και την επικοινωνία μεταξύ μελών της ομάδας ανάπτυξης. Είναι εργαλείο ανάλυσης, σχεδιασμού και τεκμηρίωσης.

Πίνακες και Οντότητες

Location: Περιλαμβάνει πληροφορίες τοποθεσίας (όνομα, διεύθυνση, συντεταγμένες κ.ά.)

Festival: Ορισμός φεστιβάλ, έτος, εικόνα και τοποθεσία.

Stage: Σκηνές με περιγραφή και χωρητικότητα.

Event: Εκδήλωση με όνομα, ημερομηνία, stage και festival.

Artist: Καλλιτέχνες με προσωπικά στοιχεία, είδος μουσικής και social media.

Band: Συγκροτήματα με ημερομηνία ίδρυσης και social.

Artist_Band: Συσχετιστικός πίνακας μεταξύ Artist και Band (N:M).

Performance: Εμφανίσεις καλλιτεχνών, τύπος, ώρα, διάρκεια.

Visitor: Πληροφορίες επισκεπτών.

Ticket: Εισιτήρια με κατηγορίες, μέθοδο πληρωμής, event, visitor.

Review: Αξιολόγηση με βάση εισιτήριο.

Resale_queue: Ουρά μεταπώλησης εισιτηρίων.

Staff: Προσωπικό με ρόλο και επίπεδο εμπειρίας.

Employment: Συσχετισμός προσωπικού με events.

Equipment: Εξοπλισμός που τοποθετείται σε σκηνές.

Πληθηκότητα και Σχέσεις

• Location – Festival: 1:N

• Location – Stage: 1:N

• Festival – Event: 1:N

• Stage – Event: 1:N

• Event – Performance: 1:N

• Artist – Performance: 1:N

• Artist – Band: N:M μέσω Artist_Band

• Event – Ticket: 1:N

• Visitor – Ticket: 1:N

• Staff – Event: N:M μέσω Employment

• Equipment – Stage: N:M μέσω Stage_Equipment

Indexes

• Performance(artist_id)

• Performance(event_id)

• Ticket(event_id)

• Ticket(visitor_id, event_id)

• Review(ticket_id)

• Event(festival_id)

• Artist(artist_id)

Τα indexes (ευρετήρια) σε έναν πίνακα βάσης δεδομένων είναι δομές που επιταχύνουν την αναζήτηση και την προσπέλαση των δεδομένων, με τρόπο παρόμοιο με το ευρετήριο στο τέλος ενός βιβλίου. Ας δούμε αναλυτικά:

Λειτουργία των Indexes

Ένα index δημιουργείται πάνω σε μία ή περισσότερες στήλες ενός πίνακα και επιτρέπει στον μηχανισμό της βάσης να βρίσκει πιο γρήγορα τις εγγραφές που πληρούν κάποια συνθήκη.

Χωρίς index:

- Η βάση πρέπει να σκανάρει γραμμή-γραμμή για να βρει το αποτέλεσμα. Αυτό έχει ως αποτέλεσμα την κατανάλωση αποθηκευτικού χώρου γιατί τα indexes αποθηκεύονται ξεχωριστά από τον πίνακα (άρα καταλαμβάνουν επιπλέον χώρο), καθώς επίσης και την επιβάρυνση σε εγγραφές (INSERT), διαγραφές (DELETE) και ενημερώσεις (UPDATE), διότι κάθε φορά που αλλάζει μια τιμή που είναι σε index, πρέπει να ενημερωθεί και το ευρετήριο

Με index:

- Η βάση χρησιμοποιεί μια δομή δεδομένων για να βρει γρήγορα τις αντίστοιχες γραμμές. Αυτό έχει ως αποτέλεσμα οι ερωτήσεις τύπου SELECT ... WHERE να γίνονται πολύ πιο γρήγορες και επίσης να έχουμε και καλύτερες επιδόσεις σε JOIN και ORDER BY

DDL

```
CREATE DATABASE db1;

USE db1;

CREATE TABLE Location (

    location_id INT AUTO_INCREMENT,

    name VARCHAR(255) NOT NULL,

    address VARCHAR(255),

    city VARCHAR(100),

    country VARCHAR(100),

    continent VARCHAR(50),

    latitude DECIMAL(11,8),

    longitude DECIMAL(11,8),

    PRIMARY KEY(location_id)

);


CREATE TABLE Festival (

    festival_id INT AUTO_INCREMENT,

    name VARCHAR(255) NOT NULL,

    festival_year INT NOT NULL,

    festival_image LONGBLOB,

    location_id INT,

    PRIMARY KEY(festival_id),

    FOREIGN KEY(location_id) REFERENCES Location(location_id) ON DELETE RESTRICT

);


CREATE TABLE Stage (

    stage_id INT AUTO_INCREMENT,

    name VARCHAR(255) NOT NULL,

    description TEXT,
```

```
    capacity INT NOT NULL,
    location_id INT,
    PRIMARY KEY(stage_id),
    FOREIGN KEY(location_id) REFERENCES Location(location_id) ON DELETE RESTRICT
);

CREATE TABLE Event (
    event_id INT AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    date_time DATETIME NOT NULL,
    festival_id INT,
    stage_id INT,
    PRIMARY KEY(event_id),
    FOREIGN KEY(festival_id) REFERENCES Festival(festival_id) ON DELETE CASCADE,
    FOREIGN KEY(stage_id)   REFERENCES Stage(stage_id)     ON DELETE RESTRICT
);

CREATE TABLE Artist (
    artist_id INT AUTO_INCREMENT,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    stage_name VARCHAR(255),
    birth_date DATE,
    genre VARCHAR(255),
    sub_genre VARCHAR(255),
    instagram VARCHAR(255),
    website VARCHAR(255),
    artist_image LONGBLOB,
    PRIMARY KEY(artist_id)
```

```
);

CREATE TABLE Band (
    band_id INT AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    formation_date DATE,
    instagram VARCHAR(255),
    website VARCHAR(255),
    PRIMARY KEY(band_id)
);

CREATE TABLE Artist_Band (
    artist_id INT,
    band_id   INT,
    PRIMARY KEY(artist_id, band_id),
    FOREIGN KEY(artist_id) REFERENCES Artist(artist_id) ON DELETE CASCADE,
    FOREIGN KEY(band_id)   REFERENCES Band(band_id)     ON DELETE CASCADE
);

CREATE TABLE Performance_Type (
    type_id INT AUTO_INCREMENT,
    name    VARCHAR(50),
    PRIMARY KEY(type_id)
);

CREATE TABLE Performance (
    performance_id INT AUTO_INCREMENT,
    type_id        INT,
    start_time     DATETIME NOT NULL,
```

```
    duration       INT CHECK(duration BETWEEN 1 AND 180),
    event_id       INT,
    artist_id      INT,
    band_id        INT,
    PRIMARY KEY(performance_id),

    FOREIGN KEY(type_id)   REFERENCES Performance_Type(type_id) ON DELETE RESTRICT,
    FOREIGN KEY(event_id)  REFERENCES Event(event_id)           ON DELETE CASCADE,
    FOREIGN KEY(artist_id) REFERENCES Artist(artist_id)         ON DELETE CASCADE,
    FOREIGN KEY(band_id)   REFERENCES Band(band_id)             ON DELETE CASCADE,

    CONSTRAINT chk_artist_or_band
      CHECK (
        (artist_id IS NOT NULL AND band_id IS NULL)
        OR
        (artist_id IS NULL     AND band_id IS NOT NULL)
      )
);

CREATE TABLE Visitor (
    visitor_id   INT AUTO_INCREMENT,
    first_name   VARCHAR(255) NOT NULL,
    last_name    VARCHAR(255) NOT NULL,
    age          INT CHECK(age >= 0),
    contact_info VARCHAR(255),
    PRIMARY KEY(visitor_id)
);

CREATE TABLE Ticket_Category (
```

```
    category_id INT AUTO_INCREMENT,
    name        VARCHAR(50),
    PRIMARY KEY(category_id)
);

CREATE TABLE Payment_Method (
    payment_method_id INT AUTO_INCREMENT,
    name              VARCHAR(50),
    PRIMARY KEY(payment_method_id)
);

CREATE TABLE Ticket (
    ticket_id         INT AUTO_INCREMENT,
    category_id       INT,
    purchase_date     DATETIME NOT NULL,
    price             DECIMAL(10,2) NOT NULL,
    payment_method_id INT,
    EAN13_code        VARCHAR(13) UNIQUE NOT NULL,
    is_activated      BOOLEAN DEFAULT FALSE,
    event_id          INT,
    visitor_id        INT,
    PRIMARY KEY(ticket_id),
    FOREIGN KEY(category_id)       REFERENCES Ticket_Category(category_id) ON DELETE RESTRICT,
    FOREIGN KEY(payment_method_id) REFERENCES Payment_Method(payment_method_id) ON DELETE
RESTRICT,
    FOREIGN KEY(event_id)          REFERENCES Event(event_id)          ON DELETE CASCADE,
    FOREIGN KEY(visitor_id)        REFERENCES Visitor(visitor_id)      ON DELETE RESTRICT
);

CREATE TABLE Review (
```

```
    review_id             INT AUTO_INCREMENT,
    performance_score     INT CHECK(performance_score BETWEEN 1 and 5),
    organization_score    INT CHECK(organization_score BETWEEN 1 and 5),
    sound_lighting_score  INT CHECK(sound_lighting_score BETWEEN 1 and 5),
    stage_presence_score  INT CHECK(stage_presence_score BETWEEN 1 and 5),
    overall_score         INT CHECK(overall_score BETWEEN 1 and 5),
    ticket_id             INT,
    PRIMARY KEY(review_id),
    FOREIGN KEY(ticket_id) REFERENCES Ticket(ticket_id) ON DELETE RESTRICT
);

CREATE TABLE Resale_Status (
    status_id INT AUTO_INCREMENT,
    name      VARCHAR(50),
    PRIMARY KEY(status_id)
);

CREATE TABLE Resale_queue (
    resale_id  INT AUTO_INCREMENT,
    status_id  INT DEFAULT 1,
    queue_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ticket_id  INT UNIQUE,
    PRIMARY KEY(resale_id),
    FOREIGN KEY(status_id) REFERENCES Resale_Status(status_id) ON DELETE RESTRICT,
    FOREIGN KEY(ticket_id) REFERENCES Ticket(ticket_id)      ON DELETE CASCADE
);

CREATE TABLE Staff_Role (
    role_id INT AUTO_INCREMENT,
```

```
    name    VARCHAR(50),
    PRIMARY KEY(role_id)
);

CREATE TABLE Experience_Level (
    experience_level_id INT AUTO_INCREMENT,
    degree              INT,
    PRIMARY KEY(experience_level_id)
);

CREATE TABLE Staff (
    staff_id            INT AUTO_INCREMENT,
    first_name          VARCHAR(255) NOT NULL,
    last_name           VARCHAR(255) NOT NULL,
    age                 INT CHECK(age >= 18),
    role_id             INT,
    experience_level_id  INT,
    PRIMARY KEY(staff_id),
    FOREIGN KEY(role_id)            REFERENCES Staff_Role(role_id)       ON DELETE RESTRICT,
    FOREIGN KEY(experience_level_id)  REFERENCES Experience_Level(experience_level_id) ON DELETE
RESTRICT
);

CREATE TABLE Employment (
    staff_id INT,
    event_id INT,
    PRIMARY KEY(staff_id, event_id),
    FOREIGN KEY(staff_id) REFERENCES Staff(staff_id) ON DELETE CASCADE,
    FOREIGN KEY(event_id) REFERENCES Event(event_id) ON DELETE CASCADE
);
```

```
CREATE TABLE Equipment_Type (
    equipment_type_id INT AUTO_INCREMENT,
    name              VARCHAR(50),
    PRIMARY KEY(equipment_type_id)
);

CREATE TABLE Equipment (
    equipment_id      INT AUTO_INCREMENT,
    name              VARCHAR(255) NOT NULL,
    equipment_type_id INT,
    description       TEXT,
    quantity          INT NOT NULL CHECK(quantity>=1),
    PRIMARY KEY(equipment_id),
    FOREIGN KEY(equipment_type_id) REFERENCES Equipment_Type(equipment_type_id) ON DELETE
RESTRICT
);

CREATE TABLE Stage_Equipment (
    stage_id     INT,
    equipment_id INT,
    quantity     INT NOT NULL CHECK(quantity>=1),
    PRIMARY KEY(stage_id, equipment_id),
    FOREIGN KEY(stage_id)     REFERENCES Stage(stage_id)     ON DELETE RESTRICT,
    FOREIGN KEY(equipment_id) REFERENCES Equipment(equipment_id) ON DELETE RESTRICT
);

CREATE INDEX idx_performance_artist       ON Performance(artist_id);
CREATE INDEX idx_performance_event        ON Performance(event_id);
CREATE INDEX idx_ticket_event             ON Ticket(event_id);
```

```
CREATE INDEX idx_review_ticket           ON Review(ticket_id);

CREATE INDEX idx_artist_id               ON Artist(artist_id);

CREATE INDEX idx_ticket_visitor_event     ON Ticket(visitor_id, event_id);

CREATE INDEX idx_event_festival          ON Event(festival_id);


-- Stored Procedures & Triggers


-- 1) Procedure staff_validation

DELIMITER $$

CREATE PROCEDURE staff_validation(IN e_id INT)

BEGIN

    DECLARE total_capacity INT DEFAULT 0;

    DECLARE required_security INT;

    DECLARE required_support INT;

    DECLARE actual_security INT;

    DECLARE actual_support INT;


    SELECT SUM(s.capacity) INTO total_capacity

    FROM Event e

    JOIN Stage s ON e.stage_id = s.stage_id

    WHERE e.event_id = e_id;


    SET required_security = CEIL(total_capacity * 0.05);

    SET required_support  = CEIL(total_capacity * 0.02);


    SELECT COUNT(*) INTO actual_security

    FROM Employment em

    JOIN Staff st      ON em.staff_id = st.staff_id

    JOIN Staff_Role sr ON st.role_id = sr.role_id
```

```
    WHERE em.event_id = e_id
      AND UPPER(sr.name) = 'SECURITY';

    SELECT COUNT(*) INTO actual_support
    FROM Employment em
    JOIN Staff st      ON em.staff_id = st.staff_id
    JOIN Staff_Role sr ON st.role_id = sr.role_id
    WHERE em.event_id = e_id
      AND UPPER(sr.name) = 'SUPPORT';

    IF actual_security < required_security THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Security Staff Is Not Enough';
    END IF;

    IF actual_support < required_support THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Support Staff Is Not Enough';
    END IF;
END$$
DELIMITER ;

-- 2) Procedure Call_Staff_Validation
DELIMITER $$
CREATE PROCEDURE Call_Staff_Validation(IN n INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= n DO
        CALL staff_validation(n);
        SET i = i + 1;
    END WHILE;
```

```
END$$

DELIMITER ;


-- 3) Trigger Check_vip_ticket

DELIMITER $$

CREATE TRIGGER Check_vip_ticket

BEFORE INSERT ON Ticket

FOR EACH ROW

BEGIN

   IF EXISTS (

      SELECT 1

      FROM Ticket_Category tc

      WHERE tc.category_id = NEW.category_id

        AND UPPER(tc.name) = 'VIP'

   ) THEN

      IF (

         SELECT COUNT(*) + 1

         FROM Ticket t

         JOIN Event e   ON t.event_id = e.event_id

         JOIN Stage s   ON e.stage_id = s.stage_id

         JOIN Ticket_Category tc ON t.category_id = tc.category_id

         WHERE UPPER(tc.name) = 'VIP'

           AND e.stage_id = (

               SELECT stage_id

               FROM Event

               WHERE event_id = NEW.event_id

           )

      ) > (

         SELECT CEIL(s.capacity * 0.1)
```

```
            FROM Event e
            JOIN Stage s ON e.stage_id = s.stage_id
            WHERE e.event_id = NEW.event_id
        ) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'VIP ticket limit exceeded';
        END IF;
    END IF;
END$$
DELIMITER ;

-- 4) Trigger Check_stage_capacity
DELIMITER $$
CREATE TRIGGER Check_stage_capacity
BEFORE INSERT ON Ticket
FOR EACH ROW
BEGIN
    DECLARE v_stage_id     INT;
    DECLARE v_capacity     INT;
    DECLARE v_sold_tickets INT;

    SELECT e.stage_id INTO v_stage_id
    FROM Event e
    WHERE e.event_id = NEW.event_id;

    SELECT s.capacity INTO v_capacity
    FROM Stage s
    WHERE s.stage_id = v_stage_id;

    SELECT COALESCE(SUM(IF(is_activated,1,0)),0) INTO v_sold_tickets
```

```
    FROM Ticket t
    WHERE t.event_id = NEW.event_id;

    IF v_sold_tickets + 1 > v_capacity THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot sell more tickets. Capacity is over.';
    END IF;
END$$
DELIMITER ;

DELIMITER $$

CREATE TRIGGER Check_duplicate_ticket
BEFORE INSERT ON Ticket
FOR EACH ROW
BEGIN
    DECLARE event_date DATE;
    DECLARE existing_ticket_id INT DEFAULT NULL;

    -- Λήψη ημερομηνίας του event
    SELECT DATE(e.date_time)
    INTO event_date
    FROM Event e
    WHERE e.event_id = NEW.event_id;

    -- Έλεγχος για διπλό εισιτήριο
    SELECT t.ticket_id
    INTO existing_ticket_id
    FROM Ticket t
    JOIN Event e2 ON t.event_id = e2.event_id
```

```
    WHERE t.visitor_id = NEW.visitor_id
      AND DATE(e2.date_time) = event_date
    LIMIT 1;

    -- Εμφάνιση γενικού σφάλματος (χωρίς δυναμικές τιμές)
    IF existing_ticket_id IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Duplicate ticket for same visitor on the same day.';
    END IF;
END$$

DELIMITER ;

-- 6) Trigger Check_artist_double_performance
DELIMITER $$
CREATE TRIGGER Check_artist_double_performance
BEFORE INSERT ON Performance
FOR EACH ROW
BEGIN
    DECLARE event_dt DATETIME;
    SELECT e.date_time INTO event_dt
    FROM Event e
    WHERE e.event_id = NEW.event_id;

    IF EXISTS (
        SELECT 1
        FROM Performance p
        JOIN Event e2 ON p.event_id = e2.event_id
        WHERE p.artist_id = NEW.artist_id
```

```
        AND e2.stage_id <> (
            SELECT stage_id FROM Event WHERE event_id = NEW.event_id
        )
        AND e2.date_time = event_dt
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Artist cannot perform on two stages at the same time';
    END IF;
END$$
DELIMITER ;


-- 7) Trigger No_4th_continuous_year (παράδειγμα για καλλιτέχη ή συγκρότημα)
DELIMITER $$
CREATE TRIGGER No_4th_continuous_year
BEFORE INSERT ON Performance
FOR EACH ROW
BEGIN
    DECLARE v_year INT;
    SELECT f.festival_year INTO v_year
    FROM Event e
    JOIN Festival f ON e.festival_id = f.festival_id
    WHERE e.event_id = NEW.event_id;


    IF NEW.artist_id IS NOT NULL THEN
        IF EXISTS(
            SELECT 1
            FROM Performance p
            JOIN Event e1 ON p.event_id = e1.event_id
            JOIN Festival f1 ON e1.festival_id = f1.festival_id
            WHERE p.artist_id = NEW.artist_id
```

```
        AND f1.festival_year = v_year - 1
    )
    AND EXISTS(
        SELECT 1
        FROM Performance p
        JOIN Event e2 ON p.event_id = e2.event_id
        JOIN Festival f2 ON e2.festival_id = f2.festival_id
        WHERE p.artist_id = NEW.artist_id
        AND f2.festival_year = v_year - 2
    )
    AND EXISTS(
        SELECT 1
        FROM Performance p
        JOIN Event e3 ON p.event_id = e3.event_id
        JOIN Festival f3 ON e3.festival_id = f3.festival_id
        WHERE p.artist_id = NEW.artist_id
        AND f3.festival_year = v_year - 3
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Artist cannot perform for 4th continuous year';
    END IF;
END IF;

IF NEW.band_id IS NOT NULL THEN
    IF EXISTS(
        SELECT 1
        FROM Performance p
        JOIN Event e1 ON p.event_id = e1.event_id
        JOIN Festival f1 ON e1.festival_id = f1.festival_id
        WHERE p.band_id = NEW.band_id
```

```
            AND f1.festival_year = v_year - 1
        )
        AND EXISTS(
            SELECT 1
            FROM Performance p
            JOIN Event e2 ON p.event_id = e2.event_id
            JOIN Festival f2 ON e2.festival_id = f2.festival_id
            WHERE p.band_id = NEW.band_id
              AND f2.festival_year = v_year - 2
        )
        AND EXISTS(
            SELECT 1
            FROM Performance p
            JOIN Event e3 ON p.event_id = e3.event_id
            JOIN Festival f3 ON e3.festival_id = f3.festival_id
            WHERE p.band_id = NEW.band_id
              AND f3.festival_year = v_year - 3
        ) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Band cannot perform for 4th continuous year';
        END IF;
    END IF;
END$$
DELIMITER ;


DELIMITER $$


DELIMITER $$


DELIMITER $$
```

```
CREATE PROCEDURE Call_All_Staff_Validation()

BEGIN

  DECLARE ev_id INT DEFAULT 0;

  DECLARE max_e INT;



  SELECT MAX(event_id) INTO max_e FROM Event;

  WHILE ev_id < max_e DO

    SET ev_id = ev_id + 1;

    CALL staff_validation(ev_id);

  END WHILE;

END$$

DELIMITER ;
```

Procedures

• Staff_Validation: Ελέγχει αν υπάρχουν αρκετοί υπάλληλοι ασφαλείας και υποστήριξης για event.

• Call_Staff_Validation: Εκτελεί την παραπάνω διαδικασία για n events.

Οι stored procedures είναι συναρτήσεις SQL που αποθηκεύονται στη βάση δεδομένων και εκτελούνται στον ίδιο τον server της βάσης.

Γιατί χρησιμοποιούμε DELIMITER:

Η SQL έχει ως προκαθορισμένο τελείωμα εντολής το ;. Όμως, μέσα σε stored procedures ή triggers χρησιμοποιείς πολλά ; ενδιάμεσα. Γι' αυτό πρέπει να ορίσεις ένα νέο προσωρινό delimiter ώστε η βάση να καταλάβει πού τελειώνει όλη η εντολή CREATE PROCEDURE, CREATE TRIGGER κ.λπ.

Triggers

• Check_Vip_Ticket: Ελέγχει αν τα VIP εισιτήρια υπερβαίνουν το 10% της χωρητικότητας.

• Check_Stage_Capacity: Αποτρέπει υπέρβαση χωρητικότητας σκηνής.

• Check_Duplicate_Ticket: Αποτρέπει έκδοση πολλών εισιτηρίων την ίδια μέρα για ίδιο επισκέπτη.

• Check_Artist_Double_Performance: Ελέγχει αν καλλιτέχνης παίζει σε 2 σκηνές ταυτόχρονα.

• No_4th_conc: Αποτρέπει καλλιτέχνη να παίζει 4η συνεχόμενη χρονιά.

• Trg_Prevent_Stage_Overlap: Ελέγχει αν υπάρχει χρονική επικάλυψη εμφάνισης σε σκηνή.

Τα triggers είναι ειδικά κομμάτια SQL κώδικα που εκτελούνται αυτόματα όταν συμβαίνει κάποιο γεγονός (όπως INSERT, UPDATE, DELETE) σε έναν πίνακα. Μπορούν να ενεργοποιηθούν είτε πριν (π.χ. BEFORE INSERT) είτε μετά (π.χ. AFTER DELETE) από μία ενέργεια. Μας προσφέρουν αυτόματο έλεγχο και επικύρωση δεδομένων, αυτόματη ενημέρωση άλλων πινάκων και μας αποτρέπουν από μη επιθυμητές ενέργειες.

Queries

Q04.sql:

```
USE db1;

SELECT
   a.artist_id,
   a.first_name,
   a.last_name,
   AVG(r.performance_score) AS avg_performance_score,
   AVG(r.overall_score) AS avg_overall_score
FROM Artist a
JOIN Performance p ON a.artist_id = p.artist_id
JOIN Ticket t ON p.event_id = t.event_id AND t.visitor_id IS NOT NULL
JOIN Review r ON r.ticket_id = t.ticket_id
WHERE a.artist_id = 1
GROUP BY a.artist_id, a.first_name, a.last_name
ORDER BY a.last_name, a.first_name;
```

Result Grid Filter Rows: Search Export:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	a	const	PRIMARY,idx_artist_id	PRIMARY	4	const	1	
1	SIMPLE	p	ref	idx_performance_artist,idx_performance_event	idx_performance_artist	5	const	7	Using where
1	SIMPLE	t	ref	PRIMARY,idx_ticket_event,idx_ticket_visitor_event	idx_ticket_event	5	db1.p.event_id	1	Using where
1	SIMPLE	r	ref	idx_review_ticket	idx_review_ticket	5	db1.t.ticket_id	1	

Q04_index.sql:

```
USE db1;

SELECT
   a.artist_id,
   a.first_name,
   a.last_name,
   AVG(r.performance_score) AS avg_performance_score,
   AVG(r.overall_score) AS avg_overall_score
FROM Artist a
JOIN Performance p FORCE INDEX (idx_performance_artist)
   ON a.artist_id = p.artist_id
JOIN Ticket t FORCE INDEX (idx_ticket_event)
   ON p.event_id = t.event_id AND t.visitor_id IS NOT NULL
JOIN Review r FORCE INDEX (idx_review_ticket)
   ON r.ticket_id = t.ticket_id
WHERE a.artist_id = 1
GROUP BY a.artist_id, a.first_name, a.last_name
ORDER BY a.last_name, a.first_name;
```

Result Grid Filter Rows: Search Export:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	a	const	PRIMARY,idx_artist_id	PRIMARY	4	const	1	
1	SIMPLE	p	ref	idx_performance_artist,idx_performance_event	idx_performance_artist	5	const	7	Using where
1	SIMPLE	t	ref	PRIMARY,idx_ticket_event,idx_ticket_visitor_event	idx_ticket_event	5	db1.p.event_id	1	Using where
1	SIMPLE	r	ref	idx_review_ticket	idx_review_ticket	5	db1.t.ticket_id	1	

traces:

Αναλύσαμε το σχέδιο εκτέλεσης δύο ερωτημάτων SQL - ένα με default index και ένα με χρήση FORCE INDEX σε συνδέσεις κλειδιών (Performance, Ticket, Review). Και τα δύο σχέδια χρησιμοποίησαν τα ίδια ευρετήρια: idx_performance_artist, idx_ticket_event και idx_review_ticket, με τύπο πρόσβασης ref και χαμηλές εκτιμήσεις γραμμών (1 έως 7 γραμμές). Αυτό υποδεικνύει ότι ο βελτιστοποιητής έχει ήδη επιλέξει τις πιο αποδοτικές διαδρομές. Ως αποτέλεσμα, η επιβολή του ευρετηρίου δεν βελτίωσε την απόδοση στη συγκεκριμένη περίπτωση, αλλά χρησίμευσε για να επιβεβαιώσει ότι ο σχεδιασμός του ευρετηρίου ήταν ορθός και αξιοποιήθηκε αποτελεσματικά. Έτσι, ενώ το FORCE INDEX μπορεί να είναι χρήσιμο σε πιο σύνθετα ή ασαφή ερωτήματα, εδώ δεν είχε κανένα μετρήσιμο αποτέλεσμα.

Q06.sql:

```
USE db1;

SELECT
   e.event_id,
   e.name AS event_name,
   f.name AS festival_name,
   f.festival_year,
   AVG(r.overall_score) AS avg_review_score
FROM Ticket t
JOIN Event e ON t.event_id = e.event_id
JOIN Festival f ON e.festival_id = f.festival_id
JOIN Review r ON r.ticket_id = t.ticket_id
```

```
WHERE t.visitor_id = 2
GROUP BY e.event_id, e.name, f.name, f.festival_year
ORDER BY f.festival_year DESC, f.name, e.name;
```

Result Grid Filter Rows: Search Export:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	r	ALL	idx_review_ticket	NULL	NULL	NULL	22	Using where; Using temporary; Using filesort
1	SIMPLE	f	eq_ref	PRIMARY	PRIMARY	4	db1.e.festival_id	1	
1	SIMPLE	e	eq_ref	PRIMARY,idx_event_festival	PRIMARY	4	db1.t.event_id	1	Using where
1	SIMPLE	t	eq_ref	PRIMARY,idx_ticket_event,idx_ticket_visitor_event	PRIMARY	4	db1.r.ticket_id	1	Using where

Q06_index.sql

```
USE db1;

EXPLAIN
SELECT
   e.event_id,
   e.name AS event_name,
   f.name AS festival_name,
   f.festival_year,
   AVG(r.overall_score) AS avg_review_score
FROM Ticket t FORCE INDEX (idx_ticket_visitor_event)
JOIN Event e ON t.event_id = e.event_id
JOIN Festival f ON e.festival_id = f.festival_id
JOIN Review r FORCE INDEX (idx_review_ticket) ON r.ticket_id = t.ticket_id
WHERE t.visitor_id = 2
GROUP BY e.event_id, e.name, f.name, f.festival_year
ORDER BY f.festival_year DESC, f.name, e.name;
```

id	select...	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	t	ref	idx_ticket_visitor_event	idx_ticket_visitor_event	5	const	7	Using where; Using index; Using temporary; Using filesort
1	SIMPLE	r	ref	idx_review_ticket	idx_review_ticket	5	db1.t.ticket_id	1	
1	SIMPLE	e	eq_ref	PRIMARY,idx_event_festival	PRIMARY	4	db1.t.event_id	1	Using where
1	SIMPLE	f	eq_ref	PRIMARY	PRIMARY	4	db1.e.festival_id	1	

traces:

Σε αυτό το ερώτημα, η χρήση του FORCE INDEX βελτίωσε σημαντικά το σχέδιο εκτέλεσης. Χωρίς FORCE INDEX, η MySQL εκτελούσε πλήρη σάρωση στον πίνακα Review (τύπος = ALL), αγνοούσε το διαθέσιμο ευρετήριο idx_review_ticket και παρήγαγε δαπανηρές λειτουργίες όπως η χρήση προσωρινών και η χρήση filesort. Αντίθετα, όταν εφαρμόστηκε FORCE INDEX, το σχέδιο εκτέλεσης χρησιμοποίησε μια ref πρόσβαση στο Review, σαρώνοντας μόνο μια γραμμή με το κατάλληλο ευρετήριο. Αυτό μείωσε την επιβάρυνση και αφαίρεσε την ανάγκη για προσωρινή ταξινόμηση. Επιπλέον, η επιβολή του index, idx_ticket_visitor_event στο Ticket εξασφάλισε μια πιο ελεγχόμενη διαδρομή πρόσβασης, παρόλο που το όφελος ήταν πιο μέτριο. Συνολικά, το ερώτημα με FORCE INDEX είναι σαφώς πιο βελτιστοποιημένο και αποφεύγει ακριβές λειτουργίες που προκαλούνται από την αρχική λανθασμένη εκτίμηση του βελτιστοποιητή.